

A Controlled Ablation Study of Learning Rate, Optimizer, and Batch Size in CNN-Based Binary Fashion-MNIST Classification

Final Course Project – Deep Learning (Semester B, 2026)

David Azimov

Department of Intelligent Systems
Engineering (M.Sc.)
Afeka - The Academic College of
Engineering in Tel Aviv
Tel Aviv, Israel
davidazimov069@gmail.com

Aviad Tzafri

Department of Intelligent Systems
Engineering (M.Sc.)
Afeka - The Academic College of
Engineering in Tel Aviv
Tel Aviv, Israel
aviadzafri@gmail.com

Abstract—This study investigates the influence of three fundamental training hyperparameters - learning rate, optimizer, and batch size - on the performance of a convolutional neural network (CNN) for binary Fashion-MNIST classification. A controlled ablation methodology was adopted in which only one hyperparameter was modified at a time while all other training conditions remained unchanged. The experiments evaluated convergence behavior, classification performance, and internal weight dynamics using validation loss, standard classification metrics, and layer-wise weight-change statistics. The results show that the learning rate had the greatest impact on training stability and predictive performance, with a learning rate of 0.001 providing the most consistent results. Adam achieved the best overall balance between convergence speed and classification accuracy, while batch size had a comparatively smaller influence within the evaluated range. These findings highlight the relative importance of common training hyperparameters and demonstrate the value of combining external performance measures with internal optimization analysis.

Keywords— Convolutional Neural Network (CNN), Fashion-MNIST, Hyperparameter Optimization, Ablation Study, Learning Rate, Optimizer, Batch Size.

I. INTRODUCTION

The performance of a convolutional neural network (CNN) is influenced not only by its architecture but also by the selection of training hyperparameters. Among these, the learning rate, optimization algorithm, and batch size have a direct impact on convergence, training stability, and predictive performance. Although these hyperparameters are routinely tuned during model development, their individual contributions are often difficult to evaluate because multiple parameters are frequently modified simultaneously.

This study presents a controlled ablation analysis in which only one hyperparameter is varied at a time while all other training conditions remain unchanged. Such an approach enables a systematic comparison of the influence of each hyperparameter on both the optimization process and the final model performance. The experiments were conducted using a fixed CNN architecture on a binary classification task derived from the Fashion-MNIST dataset, with independent evaluations of learning rate, optimizer, and batch size.

Model performance was assessed using standard classification metrics, including Accuracy, Precision, Recall, Specificity, F1-score, ROC-AUC, and confusion matrices. In

addition, internal training dynamics were investigated by monitoring changes in network weights throughout the optimization process. Rather than proposing a new network architecture, the objective of this work is to improve the understanding of how fundamental training hyperparameters influence CNN training under controlled experimental conditions.

II. METHODOLOGY

A. Dataset

The experiments were conducted using the Fashion-MNIST dataset [1], which consists of 28×28 grayscale images of clothing items belonging to ten predefined categories. To focus the study on the influence of training hyperparameters rather than multi-class decision boundaries, the original dataset was converted into a balanced binary classification problem by selecting the T-shirt/top and Shirt classes. These categories were chosen because of their visual similarity, resulting in a more challenging classification task than classes with clearly distinguishable characteristics.

The combined dataset was randomly partitioned into training, validation, and test subsets using an 80/10/10 split while preserving class balance across all partitions. The binary formulation also simplifies the interpretation of confusion-matrix-based evaluation metrics, enabling a direct analysis of True Positives (TP), False Positives (FP), True Negatives (TN), False Negatives (FN), Accuracy, Precision, Recall, Specificity, F1-score, and ROC-AUC throughout the experimental study.

B. CNN Architecture

A single baseline Convolutional Neural Network (CNN) architecture was employed throughout all experiments to ensure that performance differences resulted exclusively from the investigated training hyperparameters. The architecture is illustrated in Fig. 1. The network receives $28 \times 28 \times 1$ grayscale images as input and consists of two convolutional blocks. The first block includes a 2D convolutional layer with 32 filters of size 3×3 , followed by a ReLU activation function and a 2×2 max-pooling layer. The second block follows the same structure, using 64 filters before a second max-pooling operation. The extracted feature maps are subsequently flattened and passed to a fully connected layer containing 64 neurons with ReLU activation. A dropout layer is applied before the final output neuron, which uses a sigmoid

activation function to produce the probability of the positive class.

The resulting network contains approximately 121,000 trainable parameters and remained unchanged throughout the entire ablation study. Maintaining a fixed architecture ensured that any observed differences in convergence behavior, classification performance, and internal weight dynamics could be attributed solely to variations in the learning rate, optimizer, or batch size rather than architectural modifications.

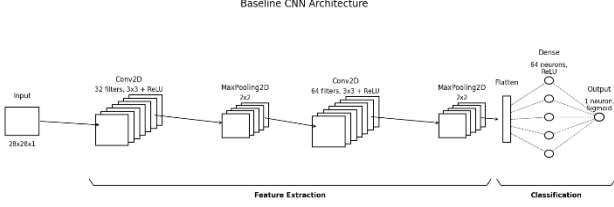


Figure 1: Architecture of the CNN used for binary Fashion-MNIST classification.

C. Experimental Protocol

To ensure a fair comparison between experimental configurations, a controlled ablation methodology was adopted in which only one training hyperparameter was modified at a time. For each experiment, a new instance of the CNN was initialized and trained using the same training, validation, and test sets, identical network architecture, loss function, and random seed. This procedure isolated the effect of the investigated hyperparameter while minimizing the influence of unrelated factors.

The baseline configuration employed the Adam optimizer [2] with a learning rate of 0.001 and a batch size of 64. Each experiment was trained for a maximum of 30 epochs using binary cross-entropy loss. Early stopping [5] monitored the validation loss with a patience of five epochs and restored the model weights corresponding to the lowest validation loss. After training, the restored model was evaluated on the held-out test set, and both external performance metrics and internal weight-change statistics were recorded for subsequent analysis.

D. Evaluation Metrics

To provide a comprehensive evaluation of each experimental configuration, three complementary categories of metrics were collected throughout the study.

The first category characterizes the training process by monitoring the training and validation loss at every epoch. In addition, the total number of training epochs and the epoch corresponding to the minimum validation loss were recorded to compare the optimization behavior of different hyperparameter configurations.

The second category evaluates the predictive performance of the trained models using standard binary classification metrics derived from the confusion matrix. These include True Positives (TP), False Positives (FP), True Negatives (TN), False Negatives (FN), Accuracy, Precision, Recall, Specificity, F1-score, and ROC-AUC.

The third category analyzes the internal behavior of the optimization process by monitoring changes in the network

weights during training. For each experiment, statistics including the mean, median, and cumulative weight changes, together with their absolute values, were computed across all trainable parameters. In addition, the most volatile weight and the three most volatile weights were identified and tracked across training epochs to examine the consistency of parameter updates under different training configurations.

Together, these three categories provide complementary perspectives on model behavior by evaluating the training process, predictive performance, and internal optimization dynamics.

III. EXPERIMENTAL SETUP

A. Learning Rate Ablation

The first ablation study investigated the influence of the learning rate on model training. Four learning rates (0.1, 0.05, 0.01, and 0.001) were evaluated while keeping the optimizer fixed as Adam and the batch size fixed at 64. The learning rate of 0.001 served as the baseline configuration. This experiment examined how the step size used during parameter updates affected convergence behavior, classification performance, and the evolution of the network weights.

B. Optimizer Ablation

The second ablation study compared four optimization algorithms: stochastic gradient descent (SGD), SGD with momentum [4], RMSprop [3], and Adam [2]. For all optimizer experiments, the learning rate was fixed at 0.001 and the batch size was fixed at 64. This configuration enabled a direct comparison of the optimization strategies while eliminating the influence of other training parameters.

C. Batch Size Ablation

The third ablation study evaluated the effect of mini-batch size by training the network with batch sizes of 16, 32, 64, and 128. Adam was used as the optimizer with a fixed learning rate of 0.001 for all experiments. The batch size of 64 corresponded to the baseline configuration. This experiment assessed how the number of training samples processed in each optimization step influenced convergence characteristics, predictive performance, and internal weight-update dynamics.

IV. RESULTS AND DISCUSSION

The ablation results are summarized in Table I. The table reports the principal convergence and predictive-performance measures for all twelve experimental configurations, while Fig. 2 presents the corresponding validation-loss trajectories. The results are analyzed separately for learning rate, optimizer selection, and batch size, followed by an analysis of the internal weight dynamics.

TABLE I. SUMMARY OF ABLATION RESULTS

Ablation	Configuration	Epochs Trained	Best Epoch	Best Val. Loss	Accuracy	F1-Score	ROC-AUC
Learning Rate	0.100	11	6	0.6933	0.5000	0.000	0.5000
	0.050	6	1	0.6938	0.5000	0.000	0.5000
	0.010	11	6	0.3183	0.8729	0.8760	0.9522
	0.001	13	8	0.3007	0.8926	0.8926	0.9613
Optimizer	SGD	30	30	0.4653	0.8114	0.8009	0.8741
	SGD+ Momentum	30	30	0.3490	0.8593	0.8602	0.9389
	RMSprop	18	13	0.2985	0.8850	0.8815	0.9621
	Adam	13	8	0.3007	0.8936	0.8926	0.9613
Batch Size	16	9	4	0.3060	0.8879	0.8876	0.9580
	32	9	4	0.3291	0.8736	0.8785	0.9555
	64	13	8	0.3007	0.8936	0.8926	0.9613
	128	17	12	0.2916	0.8979	0.8969	0.9611

A. Learning Rate Analysis

A The learning-rate ablation produced the largest differences among all investigated hyperparameters. As shown in Table I, learning rates of 0.1 and 0.05 failed to learn the classification task. Both configurations achieved an accuracy of 0.5000, an F1-score of 0, and a ROC-AUC of 0.5000. Their best validation losses remained close to 0.693, which corresponds to uninformative behavior in balanced binary classification.

The failure of these configurations is also visible in Fig. 2(a). Their validation-loss curves remained nearly constant throughout training, indicating that the model did not progress toward a discriminative solution. The corresponding confusion-matrix statistics showed that both configurations classified every test sample as the negative class, producing 700 true negatives and 700 false negatives. Their specificity was therefore 1.0, but this value was misleading because recall for the positive class was zero.

Reducing the learning rate to 0.01 enabled successful learning. This configuration achieved an accuracy of 0.8729, an F1-score of 0.8760, and a ROC-AUC of 0.9522. However, its validation-loss trajectory exhibited noticeable fluctuations after reaching its minimum, suggesting that the optimization process remained relatively sensitive to the update magnitude.

The best result in this ablation group was obtained with a learning rate of 0.001. It achieved an accuracy of 0.8936, an F1-score of 0.8926, and a ROC-AUC of 0.9613. Its validation loss decreased gradually and reached a minimum of 0.3007 at epoch 8. These results indicate that 0.001 provided the most favorable balance between optimization stability and predictive performance for the investigated architecture.

B. Optimizer Analysis

The optimizer ablation produced substantial differences in convergence and predictive performance. Standard SGD achieved the weakest results, with an accuracy of 0.8114, an F1-score of 0.8009, and a ROC-AUC of 0.8741. Its best validation result occurred at epoch 30, which was the final permitted epoch. Therefore, the model did not satisfy the

early-stopping condition and had not clearly converged within the available training budget.

Adding momentum substantially improved SGD. Accuracy increased to 0.8593, while the F1-score and ROC-AUC increased to 0.8602 and 0.9389, respectively. As shown in Fig. 2(b), momentum produced a much faster initial reduction in validation loss than standard SGD. Nevertheless, its best validation loss was also obtained at epoch 30, indicating that additional training might still have produced further improvement.

RMSprop and Adam clearly outperformed the two SGD configurations. RMSprop achieved the lowest validation loss in the optimizer group, 0.2985, and the highest ROC-AUC, 0.9621. Adam achieved slightly higher accuracy and F1-score, reaching 0.8936 and 0.8926, respectively, compared with 0.8850 and 0.8815 for RMSprop.

The difference between the two adaptive optimizers was therefore metric-dependent. RMSprop provided marginally better threshold-independent class separation, whereas Adam provided the strongest classification performance at the selected decision threshold. Adam also reached its best validation loss after eight epochs, compared with thirteen epochs for RMSprop. Adam consequently provided the best overall balance between convergence efficiency and classification performance.

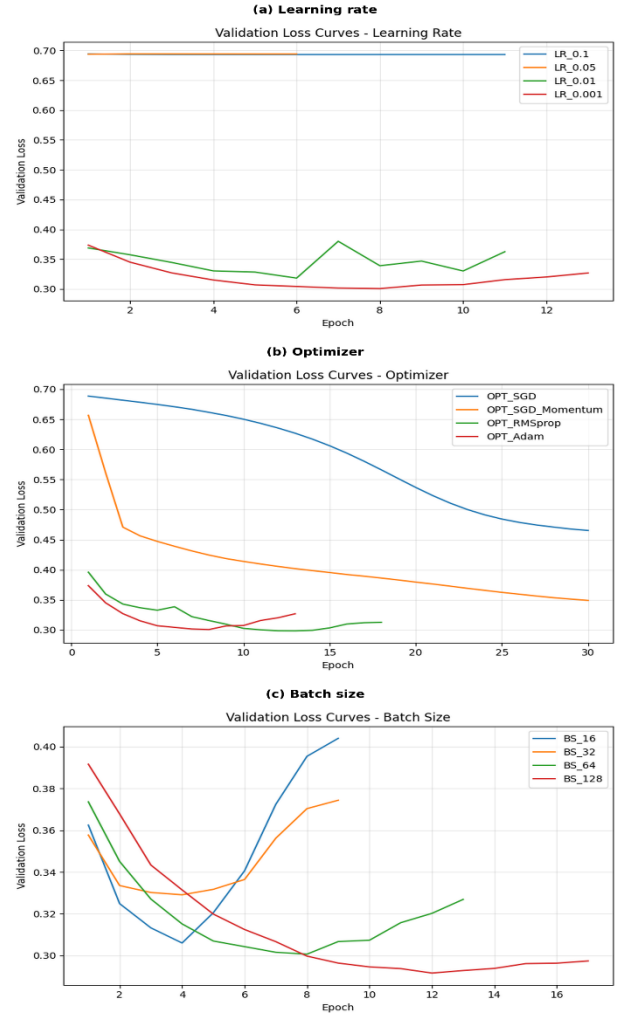


Figure 2: Validation-loss trajectories for the three ablation groups: (a) learning rate, (b) optimizer, and (c) batch size.

C. Batch Size Analysis

The batch-size ablation produced smaller differences than the learning-rate and optimizer experiments. All four batch sizes generated effective classifiers, with accuracy values between 0.8736 and 0.8979. This indicates that, within the evaluated range, batch size affected the results less severely than either learning rate or optimizer selection.

The weakest result was obtained with a batch size of 32, which achieved an accuracy of 0.8736 and an F1-score of 0.8785. Although this configuration produced relatively high positive-class recall, its lower specificity indicated a greater tendency to classify samples as Shirts, thereby increasing the number of false positives.

As shown in Fig. 2(c), batch sizes of 16 and 32 reached their minimum validation losses relatively early, after which validation performance deteriorated. A batch size of 64 produced stronger and more stable results, achieving an accuracy of 0.8936 and an F1-score of 0.8926.

The best batch-size configuration was 128. It achieved the lowest validation loss, 0.2916, together with the highest accuracy and F1-score in the complete experiment, 0.8979 and

0.8969, respectively. Although it required more epochs to reach its best result, each epoch contained fewer optimizer steps. It therefore reached its best epoch after 1,056 optimizer updates, compared with 1,400 updates for batch sizes 32 and 64 and 2,800 updates for batch size 16.

The improvement over batch size 64 was nevertheless modest, amounting to approximately 0.43 percentage points in both accuracy and F1-score. The result supports batch size 128 as the strongest tested configuration, but it should not be interpreted as evidence that larger batches are universally superior.

D. Internal Weight-Dynamics Analysis

Representative examples of the layer-wise weight dynamics are presented in Fig. 3. Unlike the external performance measures, which indicate whether a configuration produced an effective classifier, these analyses provide insight into how the optimization process evolved internally. Therefore, the epoch-level changes in the trainable weights were analyzed using their signed and absolute means, sums, medians, maximum changes, and the identities of the most volatile weights.

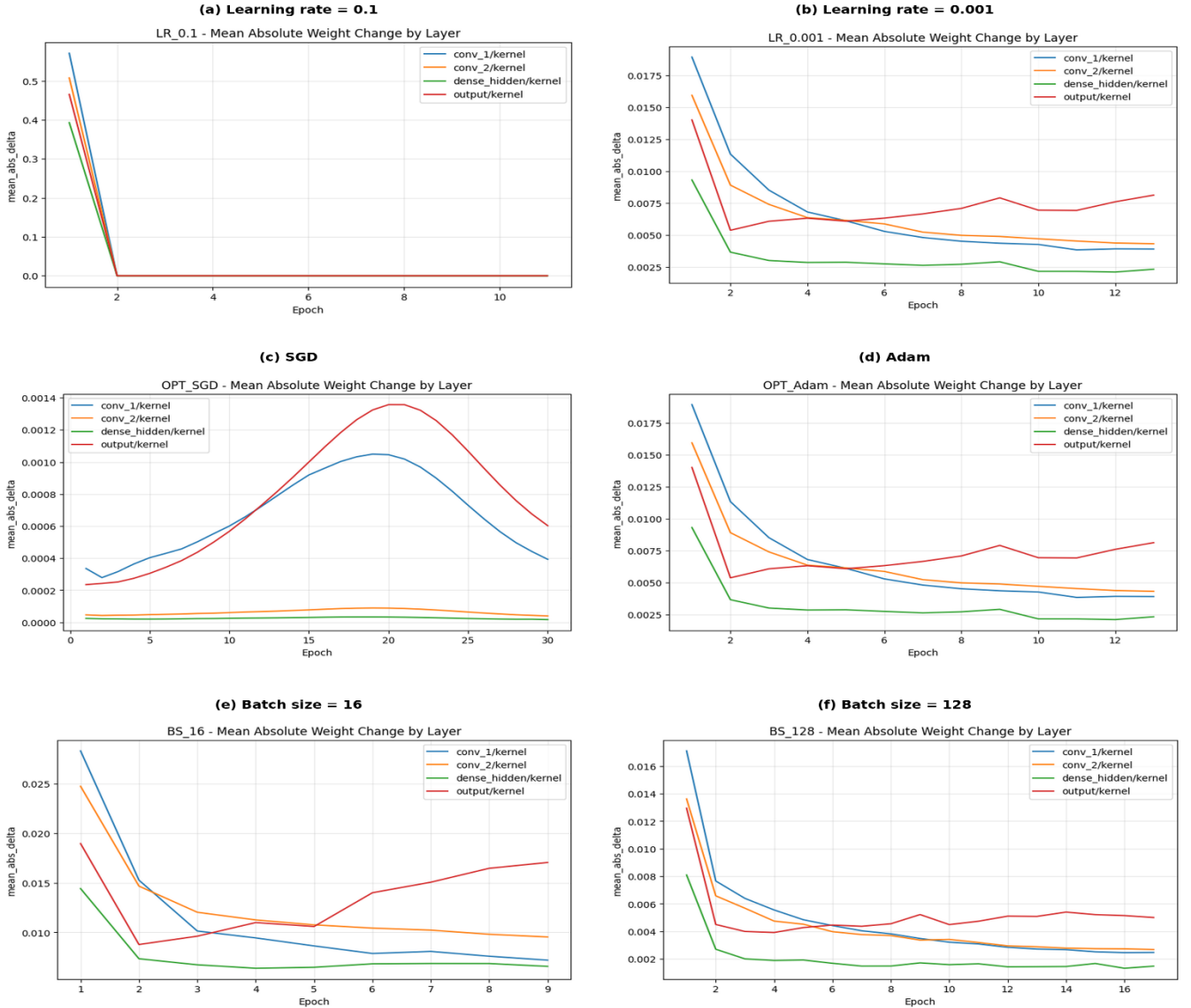


Figure 3: Mean absolute weight changes by layer for representative configurations from each ablation group: (a) learning rate = 0.1, (b) learning rate = 0.001, (c) SGD, (d) Adam, (e) batch size = 16, and (f) batch size = 128.

The clearest relationship was observed in the learning-rate experiments. Large learning rates produced substantially larger weight changes than the stable baseline configuration. For example, the mean absolute epoch-level change in the first convolutional layer was 0.051887 for a learning rate of 0.1, compared with 0.006668 for a learning rate of 0.001. Similar differences were observed in the remaining trainable layers.

Despite producing much larger parameter changes, learning rates of 0.1 and 0.05 failed to learn the classification task. Large weight movement therefore did not represent effective learning in these experiments; instead, it reflected an excessively aggressive and unstable optimization process. By contrast, the learning rate of 0.001 produced smaller, sustained changes and achieved strong predictive performance.

Standard SGD exhibited the opposite behavior. Its mean absolute weight changes were particularly small, including 0.000675 in the first convolutional layer and 0.000063 in the second convolutional layer. These small changes were accompanied by slow convergence and weaker predictive performance. Taken together, the results indicate that successful learning required updates that were large enough to produce meaningful progress but not so large that they destabilized the optimization process.

The batch-size experiments showed that epoch-level weight changes generally decreased as batch size increased. However, a direct epoch-level comparison can be misleading because smaller batches perform more optimizer updates during each epoch. For this reason, the study also normalized weight-change measures by the number of optimizer steps, allowing a fairer comparison between batch sizes.

E. Overall Findings and Limitations

Overall, learning rate had the strongest influence on the outcome of training. Values of 0.1 and 0.05 caused complete classification failure, whereas 0.01 and 0.001 enabled effective learning. Optimizer selection had the second-largest effect: Adam and RMSprop clearly outperformed SGD and SGD with momentum. Batch size produced more moderate differences, since all evaluated values resulted in comparatively strong classifiers.

The strongest tested configuration in terms of accuracy and F1-score combined Adam, a learning rate of 0.001, and a batch size of 128. It achieved an accuracy of 0.8979 and an F1-score of 0.8969. RMSprop with a batch size of 64 produced a marginally higher ROC-AUC of 0.9621, demonstrating that the optimal configuration can depend on the selected evaluation criterion.

Several limitations must be considered. The study examined only a binary subset of Fashion-MNIST and one CNN architecture. Furthermore, each configuration was trained using one fixed random seed, so variability between independent runs was not measured. Weight changes were recorded at epoch boundaries rather than after every optimizer step, meaning that conclusions regarding individual-step volatility are indirect. Finally, the original Fashion-MNIST training and test partitions were combined before constructing a new stratified 80/10/10 split. The results are therefore

appropriate for internal comparison between the tested configurations but should not be compared directly with standard benchmark results based on the original test partition.

V. CONCLUSIONS

This study investigated the influence of three fundamental training hyperparameters—learning rate, optimizer, and batch size—on the performance and training dynamics of a convolutional neural network for binary Fashion-MNIST classification. Using a controlled ablation methodology in which only one hyperparameter was modified at a time, the experiments evaluated both predictive performance and internal network behavior. This approach enabled a systematic comparison of convergence characteristics, classification metrics, and layer-wise parameter updates across different training configurations.

The experimental results demonstrated that the learning rate had the greatest impact on both convergence and final model performance. Excessively large learning rates prevented effective optimization, whereas a learning rate of 0.001 consistently produced stable convergence and high classification performance. Optimizer selection also substantially affected training behavior, with Adam providing the best overall balance between convergence speed and predictive accuracy, while RMSprop achieved comparable performance. In contrast, varying the batch size produced relatively smaller differences, indicating that the network was less sensitive to this hyperparameter under the experimental conditions. Analysis of the internal weight dynamics further supported these findings by revealing clear differences in the magnitude and evolution of parameter updates for different training configurations.

Although the study focused on a simplified binary classification task and a single CNN architecture, the results provide practical insight into the relative importance of common training hyperparameters and demonstrate the value of combining external evaluation metrics with internal optimization analysis. Future work could extend this investigation to multi-class classification, deeper network architectures, additional optimization algorithms, and automated hyperparameter optimization techniques to determine whether the observed trends generalize to more complex deep learning problems.

REFERENCES

- [1] Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A Novel Image Dataset for Benchmarking Machine Learning Algorithms*. arXiv:1708.07747.
- [2] Kingma, D. P., & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- [3] Hinton, G. (2012). *Neural Networks for Machine Learning* (Lecture 6e: RMSProp). Coursera. University of Toronto.
- [4] Qian, N. (1999). *On the Momentum Term in Gradient Descent Learning Algorithms*. *Neural Networks*, 12(1), 145-151.
- [5] Prechelt, L. (1998). *Early Stopping - But When?* In G. Orr & K.-R. Müller (Eds.), *Neural Networks: Tricks of the Trade* (pp. 55-69). Springer.